Documented departures from the official code and papers

A departure is listed here if it changes anything an official run would do. Each entry gives the reason and the evidence. No architecture, loss, sampling scheme or optimiser was changed.

A. Protocol-mandated, applied identically to the previous ten models

#	Change	Official behaviour	Why
A1	Frozen COMP8851 split registry: TR40 = 40/20/40, nested TR30–TR10, Elliptic chronological	SAGAD 100 labels, SpaceGNN 50/50, CRoC 1%	Comparability with Table 6. Our TR40 partitions equal the CARE-GNN/GHRN partitions node for node (`phase2/data.py` asserts sizes and fraud counts).
A2	Checkpoint selection on **validation AUPRC**, evaluated every epoch	SAGAD val AUROC; SpaceGNN val AUC + Macro-F1 at epoch 25; CRoC val AUC	`protocol.SELECTION_METRIC = "auprc"`
A3	Threshold sweep on validation (0.01–0.99, Macro-F1, then recall, then closeness to 0.5), applied unchanged to test	SpaceGNN and CRoC sweep 0.05–0.95 for Macro-F1; SAGAD reports none	Shared evaluator
A4	Unified budget of at most 100 epochs, early-stopping patience 20 (`--budget protocol` , the default)	SAGAD 500/50 (T-Social 1000/50); SpaceGNN 25 epochs; CRoC 200–300 epochs, no early stopping	`protocol.MAX_EPOCHS` / `EARLY_STOPPING_PATIENCE` ; CARE-GNN and GHRN were run the same way. `--budget official` reproduces the authors' budgets as a sensitivity track.
A5	Seeds 2, 42, 72; at most 12 tuning trials on TR40 with seed 2; trial 0 is the official per-dataset configuration; test never scored during tuning (`--skip-test`)	Authors' own seeds and grids	`protocol.TRAIN_SEEDS` , `MAX_TUNING_TRIALS`
A6	Canonical data: a homogeneous union view for SAGAD/SpaceGNN, and a relation view for CRoC	GADBench or DGL files	Byte-identical to the data of the previous runs (view hashes asserted).
A7	Model code receives labels for training nodes only; others are -1, or 0 together with an all-False mask for CRoC	Official scripts hold every label in memory	Makes validation/test leakage structurally impossible.

B. Adapter decisions where the official code has no answer

#	Model / data	Decision	Reason
B1	SAGAD / FDCompCN	Hyperparameters from the `default` block of `semi_train.conf.yaml`	That block is what the official code merges under any dataset.
B2	SAGAD / all	Trial 0 comes from the semi-supervised config	The fully supervised config was never released.
B3	SpaceGNN / Elliptic, FDCompCN	argparse defaults (lr 1e-3, 3 layers, dropout 0.05, α=β=0.5), stdneg = stdpos = 0.015	Not in `PARAMETERS` ; the official script would crash. 0.015 is the midpoint of the authors' sampling range.
B4	CRoC / FDCompCN	Relational branch: 3 semantic relations one-hot, min-max features, base model	`preprocess.py` routes the multi-relational graphs (yelp, amazon) this way.
B5	CRoC / Elliptic	Single-relation branch: constant 2-d edge type, z-score features, `_Large` model	`main.py` 's rule `X if dataset in ['yelp','amazon'] else X_Large` , and the tfin/tsoc preprocessing.
B6	CRoC / Elliptic, FDCompCN	main.py argparse defaults: CRoCGIN, α=0, γ=1, η=0.5	No Table 10 entry exists.

#	Model / data	Decision	Reason
B7	CRoC / T-Fin, T-Soc, Yelp	Backbone SAGE, GIN and GIN respectively	Not stated in Table 10. Inferred from the paper's results tables, or the README quick start for Yelp. The backbone is part of the tuning search space.
B8	CRoC / FDCompCN	The released `homo` edge type is excluded	It is exactly the union of the 3 semantic relations (verified in `phase2/data.py::semantic_relations`).

C. Engineering changes that do not alter the method

#	Change	Evidence of equivalence
C1	SAGAD MRQ sampling is vectorised (`mrq_one_hop_exact`): same selection rule, same float32 operations, NaN-key centres delegated to the official Python routine	`tests/test_sagad_mrq.py` checks edge-for-edge equality with the official routine on all FDCompCN nodes, on sampled Amazon and YelpChi centres, and on a synthetic graph with ties and NaN keys. It also matches the full official `mrqsample` on FDCompCN.
C2	SAGAD MRQ is computed on a copy of the features: the official cache-hit behaviour, without the in-place mutation	The test records that the official call mutates its input (`True`).
C3	SpaceGNN validation uses the paper's/test-time α/β mixture; the official validation call swaps α and β	`main.py` lines 149 vs 157; paper Algorithm 4
C4	SpaceGNN inference builds full one-hop blocks deterministically (`in_subgraph` + `to_block`); training keeps the official sampler	`tests/test_adapters_unit.py` shows the identical edge set and identical repeated predictions. Without this, one model's predictions differ by up to 4.7e-4 between two calls.
C5	SpaceGNN inference batch size is fixed at 4096 (official: val 50, test 100,000) and recorded in the config hash	Batch composition changes the hyperbolic head by about 8e-5, through float32 ill-conditioning.
C6	Execution on GPU for SpaceGNN (the official script is CPU-only)	Numerics are identical up to floating-point non-associativity.
C7	`torch.use_deterministic_algorithms(True, warn_only=True)` for SpaceGNN (official: strict)	A CUDA op without a deterministic kernel warns instead of aborting.
C8	`dgl.seed` set per epoch from (seed, epoch) for all models (CRoC never seeds DGL)	Makes runs reproducible and resumable. `tests/test_pipeline.py::test_resume_after_interruption` shows a resumed run equals an uninterrupted one.
C9	CRoC DataLoader worker count is configurable (official: 4)	Batches are drawn in the main process.
C10	Smoke tests only: induced node subsets (300,000 nodes of T-Social for every model, and 8,000 nodes of T-Finance for the local CPU SpaceGNN smoke)	Recorded as `smoke_subset_nodes` in each affected `summary.json` and shown in `audit/COMPATIBILITY_MATRIX.md`; never used for results.
C11	SAGAD MRQ graph cached on disk, keyed by the SHA256 of the exact edge list and transformed features (the official code caches `./mrqs/<dataset>`)	`tests/test_sagad_mrq.py::test_mrq_cache_returns_identical_edges` checks identical edges, a real cache hit, and a key change on different input.
C12	CRoC all-node inference scores cached until the weights change, so validation and test after loading the best checkpoint share one pass. The repeat-noise measurement invalidates the cache and recomputes.	The official evaluation scores every node in one pass, so the cache only avoids recomputing an identical pass.
C13	Opt-in only: `configs/matrix.yaml: tuning_trials_override`. It is empty by default, which is the protocol's 12 trials.	If used, it is written into that pair's `tuning_record.json` as a deviation.